

1. Voting Rule

The meeting began with clarification of the club's voting procedure.

- Elections or decisions will be determined by **the highest number of votes**, not necessarily by an absolute **majority (50%+)**.
 - In the event of a **tie**, the **president** will make the final decision.
-

2. Discussion on Game Development Scope

The group then moved into discussion of how club game projects should be structured.

- The current preference is to make **simple and manageable projects**.
 - Even if projects are smaller in scope, the goal is still to produce games with a solid UI and workable **game mechanics**.
 - The preferred engine discussed was **Unity**.
-

3. Discussion on Project Length and Structure

The next topic concerned whether the club should organize projects by semester or by longer-term development tiers.

- One proposal was **one semester, one game**.
- It was noted that a project does **not necessarily need to stop at the end of a semester**.
- An earlier concept had been to create **different tiers of games**:
 - **Core members** could work on long-term projects.
 - Semester-long projects could potentially continue into longer-term development.
- However, the discussion concluded that, **for now**, the club should focus on **semester-based projects only**.

During this part of the discussion, **SAM's game** was also mentioned as a possible point of reference or current example.

4. Discussion on File Hosting and Asset Sharing

The group then discussed practical collaboration constraints, especially around file storage.

- Jayden pointed out that **GitHub will not allow 4GB files**.
- As a result, GitHub alone is not sufficient for storing large game assets.
- The group noted that **OneDrive** could be used as an alternative file-sharing solution.
- It was mentioned that there is access to **4TB of OneDrive storage**.

This implies a likely workflow where:

- **GitHub** is used for source code and project organization.
 - **OneDrive** is used for large assets and oversized files.
-

5. Club Name

The club name was then established as:

Pitt Indie Game Dev Club
Abbreviation: PIGDC

6. Repository and Project Setup

The meeting then shifted to the club's initial technical setup.

Items mentioned:

- A **GitHub repository**
- A **first commit**
- A **GitHub Project board**

This suggests the club intends to formalize its workflow through:

- a shared repository,
 - initial project bootstrapping,
 - and a task/project management board.
-

7. Asset Creation Direction

The discussion also touched on how assets should be handled.

- The engine remains **Unity**.
- There was interest in making **assets from scratch**.

This suggests a preference toward original or club-made assets rather than relying entirely on external premade resources.

8. Audio File Format Discussion

The meeting then covered audio format preferences.

- **OGG** should be used for **most audio files**
 - **WAV** should be used for **very short sound effects**
-

9. Additional Ideas Raised

At this point, I raised two possible directions for the club to consider.

9.1 Educational / Explanatory Games

- A proposal was made for **intuition-based explanatory games**
- Examples included games that help explain:
 - **Calculus 2**
 - **Physics 1**
 - other conceptual academic material

9.2 Art and Music

- A question was also raised about whether the club should intentionally include:
 - **art-related work**
 - **music-related work**

This broadened the discussion beyond coding alone and suggested a more multidisciplinary structure.

10. Smaller-Group / Later Discussion

The notes then indicate a later or narrower discussion involving:

SAM / Jayden / Gautam / Jack / Me

This may reflect either the active speakers in the latter portion of the meeting or a smaller continuation of the discussion.

11. Push-and-Pull Design Discussion

The later part of the meeting moved into a more abstract design discussion labeled as a **push and pull issue**.

From the notes, this appears to concern:

- how much control should be given to the player,
 - how many decisions should be exposed directly,
 - and how much should instead be automated by the system.
-

12. Dave' s Design Philosophy

A specific viewpoint attributed to Dave was recorded.

- Dave prefers to **do as much as possible for the player**
- He also seems to favor reducing the number of explicit choices exposed to users or developers when those choices can be handled automatically

An example given:

- Instead of asking whether the experience should be **pixelated** or **mesh-based**,
- the system should choose automatically,
- similar to how **Roblox** handles many defaults for the user.

The principle recorded here is that Dave wants things to feel:

- **intuitive**
 - **easy**
 - and low-friction for the user
-

13. Fragmented Technical / Product Notes

The notes then contain several fragmentary items:

- “Flag a canvas group”
- “Request to do that”
- “Internal Edible”
- “Expose redeeming a mesh as API”
- “flag was default was bad”
- “New way is bad”

These appear to be partial technical or product-design remarks from the discussion.

Because the surrounding context is incomplete, they are best preserved as **raw discussion fragments** rather than treated as finalized conclusions.

14. Roblox and Versioning

The final clearly interpretable note was:

- **Roblox is unversioned so NOT Git**

This appears to reflect a distinction between:

- **Roblox-style workflow**, and
- **Git / GitHub-style version control**

The implication is that Roblox' s system is not equivalent to standard Git-based versioning and collaboration.

The final note **“Post 2026”** was also recorded, but without enough surrounding context to interpret confidently.
